Authority is All You Need: Multi-Agent Conflict Resolution from Real-World Structures

Gerald Neves

Scrums.com

Abstract

Multi-agent systems typically resolve conflicts through negotiation, convergence, or optimization. These approaches assume independent agents and require iterative coordination. In practice, real-world systems do not resolve conflict this way. Decisions are determined through *authority*: who owns the truth, who has the right to decide, and which system or structure has precedence. This paper introduces *Agent Arbiter*, a deterministic primitive for resolving conflicts in multi-agent systems using explicit authority derived from real-world structures such as organizational hierarchy, contractual relationships, and system-of-record ownership. We show how authority can be modeled as a layered, domain-scoped, and bounded structure, transforming multi-agent coordination from a search problem into a selection problem. We provide architecture patterns, practical examples, and a reference standard designed for implementation rather than theoretical completeness.

1. Introduction

Multi-agent systems have advanced rapidly across reinforcement learning, distributed systems, and autonomous agents. However, most approaches to conflict resolution assume that agents must (i) negotiate, (ii) converge, or (iii) optimize until agreement is reached. **This assumption does not hold in real systems.** In companies, contracts, infrastructure, and regulated environments: conflicts are not negotiated indefinitely, decisions are not averaged, and agents do not converge. Instead, systems resolve conflict through *precedence*. As Jim Barksdale, former CEO of Netscape, famously stated: *"If we have data, let's look at data. If all we have are opinions, let's go with mine."* [7]. Extending this principle to multi-agent systems: *we go with the agent that has authority*. This paper formalizes that shift.

2. Problem

Modern multi-agent systems face three consistent issues:
(2.1) Coordination Overhead: agents require multiple steps to reach agreement;
(2.2) Ambiguity of Ownership: it is unclear which agent should decide in a given context;
(2.3) Misalignment with Real Systems: real-world authority structures are not encoded, leading to inconsistent decisions, duplicated logic across systems, and lack of auditability.

3. Core Idea

We propose: *Multi-agent conflict resolution is not a coordination problem. It is a precedence selection problem over agents.* Instead of negotiation, voting, or convergence, we define:
selected_action = argmax(authority(agent, context)), where authority is derived from real-world structures.

3.1 Notation

We introduce minimal notation to describe the system.

Sets and variables.

• $\boldsymbol{A}$: set of agents
• $\boldsymbol{P}$: set of principals (real-world identities)
• $\boldsymbol{C} \subseteq \boldsymbol{A}$: set of candidate actions in conflict
• $\boldsymbol{x} \in \boldsymbol{X}$: context (domain, constraints, active system state)

Agent to principal mapping.

$$a \rightarrow p$$

Each agent $a \in A$ maps to a principal $p \in P$.

Authority function.

$$\mathcal{A}(a, x)$$

Returns the precedence of agent a in context x.

Resolution.

$$a^* = \text{argmax}_{a \in C}\, \mathcal{A}(a, x)$$

Where a^* is the selected agent.

Domain extension.

$$\mathcal{A}(a, x, d)$$

Where $d \in D$ represents the active domain.

4. Agent Authority Model (Practical Form)

Authority is not abstract. It is derived from: (i) **hierarchy** (who manages whom), (ii) **contracts** (who has rights), (iii) **systems** (who owns truth), and (iv) **constraints** (what cannot be violated). We model authority as: $\mathcal{A}(agent, context) = hierarchy + contract + system\_of\_record + regulatory + bounded\_signals$. This is not intended as a precise mathematical model, but as a computable structure.

5. Agent Authority Layers

Authority is resolved through ordered layers: **Constitutional** (laws, non-overridable constraints), **Institutional** (organization, contracts, policy), **System** (system-of-record ownership), and **Agent** (local decisions, preferences). *Higher layers override lower layers.*

6. Domain Boundaries (Critical)

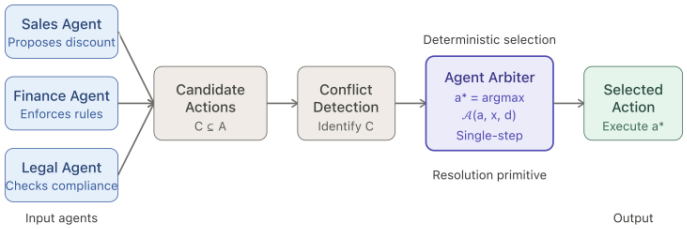
Authority is not global. It is domain-scoped. Examples of domains include: pricing, billing, contract amendments, infrastructure, and access control. An agent may have authority in one domain and none in another. Authority without domain boundaries leads to system-wide conflicts and ambiguity.

7. The 0–1 Computational Spectrum

Conflict resolution sits on a spectrum. Negotiation, reinforcement learning, and convergence are probabilistic and iterative (closer to **0**). Agent Arbiter is deterministic and single-step (closer to **1**). We define **0** as fully emergent (agents learn and negotiate) and **1** as fully deterministic (authority selects outcome). Agent Arbiter operates close to 1, with optional bounded signals introducing controlled flexibility.

8. Architecture

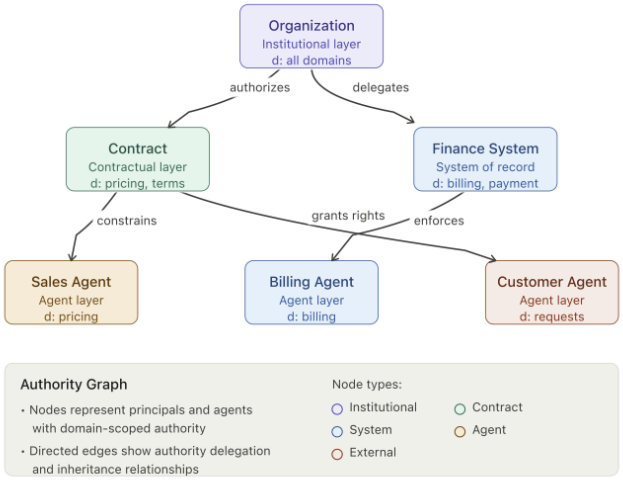
8.1 System Flow



System flow: Agents → Candidate Actions → Conflict Detection → Agent Arbiter → Execution

No negotiation • No iteration • Deterministic single-step resolution

8.2 Agent Authority Graph



9. Examples

9.1 Pricing Conflict (Business)

Agents: Sales Agent proposes discount; Billing Agent enforces pricing rules; Customer Agent requests concession. **Resolution:** authority(Sales) = 0.6, authority(Billing) = 0.9, authority(Customer) = 0.4 → selected = Billing. **No negotiation. No iteration.**

9.2 Contract Amendment Conflict

A customer requests new payment terms. **Agents:** Sales Agent accepts terms; Legal Agent enforces compliance; Finance Agent validates policy. **Outcome:** Legal authority overrides; terms constrained to compliant structure.

9.3 HVAC (Optional System Example)

User wants 16°C; system constraint requires ≥ 21°C. **Outcome:** system-of-record overrides user preference.

10. Agent Arbiter (Standard)

This paper defines the primitive implemented in the repository: **Agent Arbiter**, a deterministic component that (i) takes conflicting actions, (ii) evaluates authority, and (iii) selects a single outcome. **Core rule:** *selected = argmax(authority(agent, context))*.

11. Implementation Considerations

(11.1) Determinism: Same input must produce same output. **(11.2) Bounded Delegation:** Authority inheritance must terminate. **(11.3) Domain Scoping:** Authority must only apply within valid context. **(11.4) Explainability:** Every decision must trace to agent, principal, and authority source.

12. Relationship to Existing Work

This work aligns with and extends multi-agent reinforcement learning [3], coordination graphs [2], and game theory [4]. However, it differs in a key way: *it does not model interaction; it encodes precedence.*

13. Practical Impact

Agent Arbiter enables faster systems (no iteration), consistent decisions, auditability, and alignment with real-world operations. This is especially relevant in enterprise systems, financial systems, infrastructure, and regulated environments.

14. Conclusion

Authority is a missing primitive in multi-agent systems. By encoding real-world authority structures directly: (i) coordination becomes deterministic, (ii) systems align with reality, and (iii) complexity is reduced. Multi-agent systems do not need more negotiation. **They need clear authority.**

References

[1] Neves, G. (2026). *Authority is All You Need: Multi-Agent Conflict Resolution from Real-World Structures*. Scrums.com.

[2] Modi, P. J., Shen, W. M., Tambe, M., & Yokoo, M. (2005). *Distributed Constraint Optimization Problems*. Retrieved from https://www.cs.cmu.edu/~ggordon/10725-F12/slides/DCOP-Intro.pdf

[3] Lowe, R., Wu, Y., Tamar, A., Harb, J., Abbeel, P., & Mordatch, I. (2017). *Multi-Agent Actor-Critic for Mixed Cooperative-Competitive Environments*. arXiv:1706.02275. https://arxiv.org/abs/1706.02275

[4] Easley, D., & Kleinberg, J. (2010). *Networks, Crowds, and Markets: Reasoning About a Highly Connected World*. Cambridge University Press. https://www.cs.cornell.edu/home/kleinber/networks-book/networks-book.pdf

[5] Neves, G. (2026). *Agent Arbiter: Repository Standard and Reference Implementation*. GitHub. https://github.com/gez-scrumsdotcom/agent-arbiter

[6] Scrums.com (2026). *Software Engineering Orchestration Platform (SEOP)*. https://www.scrums.com

[7] Barksdale, J. (1990s). Attributed quote on data-driven decision making. Widely cited in business and technology contexts.

[*] Name inspired by *Attention Is All You Need*, Vaswani et al., 2017.

Appendix A: Repository Standard

A reference implementation and specification of the system described in this paper is available in the Agent Arbiter repository [5]. The repository is structured as follows:

```
agent-arbiter/
├── spec/          # authority model, layers, delegation, resolution
├── schema/        # authority graph schema
├── reference/     # minimal resolver implementation
├── examples/      # real-world scenarios
```

The repository provides:

- a minimal, composable authority model
- a schema for representing authority relationships
- a deterministic resolution primitive
- practical examples aligned with real-world systems

This appendix is intended to bridge the conceptual model presented in this paper with a directly implementable standard.

Appendix B: Core Resolution Formula

The Agent Arbiter primitive resolves conflicts through a single deterministic operation. Given a set of conflicting candidate actions C from agents in A, and a context x, the selected action is determined by:

$$selected = argmax(authority(agent, context))$$

Or more formally:

$$a^* = argmax_{a \in C} \mathcal{A}(a, x, d)$$

where a^* is the selected agent, $\mathcal{A}$ is the authority function, and d is the domain. This formula encodes the principle that *authority, not negotiation, determines outcomes*. The deterministic nature of this operation ensures: (i) reproducibility — identical inputs yield identical outputs; (ii) auditability — every decision traces to explicit authority sources; and (iii) efficiency — resolution completes in a single step without iteration.